

Data Engineering

Design Patterns

UA.DETI.DE - 2025/26

Overview

❖ What are Design Patterns?

- Reusable, battle-tested solutions to common problems in data engineering (e.g., handling small files, managing privacy, ensuring idempotency).

❖ Why use them?

- **Standardization**: Provides a common language for the team.
- **Reliability**: Reduces "reinventing the wheel" with unproven code.
- **Scalability**: Decouples architecture to handle data growth.

❖ Goal:

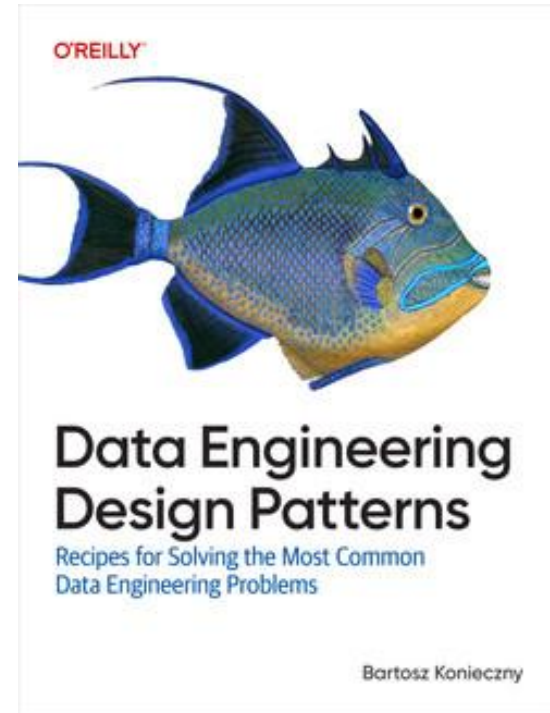
- Understand engineering enterprise-grade "systems."
- Building Robust, Scalable, and Maintainable Data Systems

Roadmap: The Pattern Catalog

- ❖ Data **Ingestion**: Strategies for moving data (Snapshot, Incremental, CDC).
- ❖ **Error** Management: The next step after the data ingestion cycle.
- ❖ **Idempotency**: Ensure consistent output without duplicates.
- ❖ Data **Value**: To augment the dataset to improve its usefulness for end users.
- ❖ Data **Flow**: Orchestrating movement and transformation (Orchestration, Streaming).
- ❖ Data **Security**: protecting assets (masking, access control, secrets).
- ❖ Data **Storage**: Organizing data for performance and cost (Bucketing, Partitioning).
- ❖ Data **Quality**: Ensuring trust and accuracy (Dead-Letter Queues, Validators).
- ❖ Data **Observability**: Monitoring health and driving ROI (FinOps, Logging).

References & Further Reading

- ❖ Primary Text:
 - Data Engineering Design Patterns –
The core foundation of these architectural principles.
- ❖ Standards & Frameworks:
 - The Apache Software Foundation: Documentation for Kafka, Airflow, and Iceberg.
 - Cloud Architecture Centers: AWS Well-Architected Framework (Data Lens), Google Cloud Data Architecture Guide, Azure Data Architecture.
- ❖ Community Resources



Data Ingestion Design Patterns



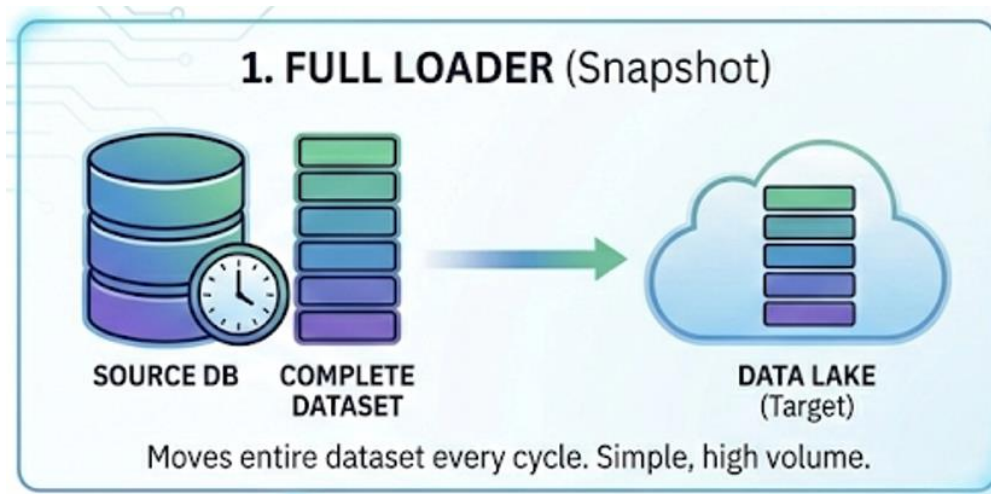
Data Ingestion Design Patterns

- ❖ **Context: The First Mile of Data Engineering**
 - Data engineering systems rarely generate their own data; they acquire it from external producers like internal teams or different organizations.
- ❖ **The Challenge:**
 - Producers have their own constraints (business/technical) and do not design their systems for your consumption.
- ❖ **The Imperative:**
 - You must adapt to these constraints to feed downstream analytics and data science workloads effectively.
- ❖ Failure to adapt leads to broken pipelines, incomplete datasets, and the need for complex backfilling operations.

Pattern 1: Full Loader (Snapshot Ingestion)

❖ Definition

- A design pattern that ingests the complete dataset during every execution cycle.
- Best suited for slowly changing dimensions (SCD), reference data (e.g., country codes), or bootstrapping a new database.



Pattern 1: Full Loader (Snapshot Ingestion)

❖ Definition

- A design pattern that ingests the complete dataset during every execution cycle.
- Best suited for slowly changing dimensions (SCD), reference data (e.g., country codes), or bootstrapping a new database.

❖ Implementation Approaches

- Passthrough (EL): Extract and Load without modification. Ideal for homogeneous stores (e.g., S3 to S3).
- ETL: Extract, Transform, Load. Required when moving between heterogeneous systems (e.g., JSON API to SQL Table) where a thin transformation layer is needed.

❖ Example

- A daily dump of "Product Categories" from an ERP system. The dataset is small (50k rows), and deletions are frequent. Full load ensures deleted categories are removed from the destination without complex logic.

Full Loader Challenges: Consistency & Volume

❖ The Consistency Hazard

- Dropping and re-inserting a table can leave consumers with partial or no data if the read happens during the write.
- Mitigation 1 (Transactions): Wrap the delete/insert in a single ACID transaction to manage data visibility.
- Mitigation 2 (View Swapping): Load data into *Table_Version_2*, then atomically update the public *View* to point to *Table_Version_2*.

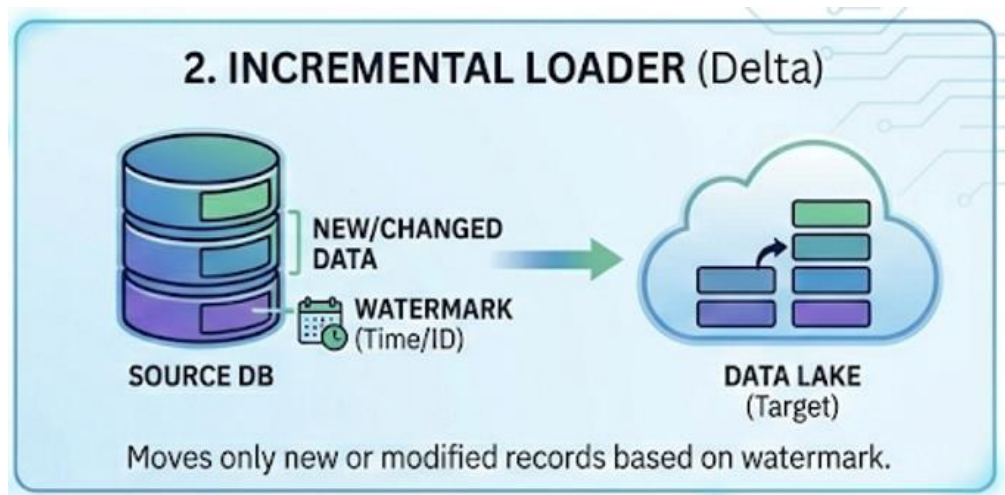
❖ The Volume Hazard

- Static infrastructure may fail if the dataset size spikes unexpectedly (e.g., doubling overnight).
- Solution: Leverage auto-scaling capabilities of the data processing layer rather than fixed-size instances.

Pattern 2: Incremental Loader

❖ Definition

- Ingests only the data that has changed or been created since the last execution.
- Essential for continuously growing datasets where a full load is cost-prohibitive.



Pattern 2: Incremental Loader

❖ Implementation: **Delta Column**

- Relies on a watermark column (e.g., `updated_at`, `ingestion_time`) to identify rows added since the last run.
- Logic: `SELECT * FROM source WHERE updated_at > last_job_run_time.`

❖ Implementation: **Partition-Based**

- Relies on storage layout (e.g., `/data/date=2023-10-27/`). The job targets specific time partitions.
- More robust for backfilling as it avoids scanning the entire table and leverages logical organization.

Incremental Loader Challenges

❖ The "Hard Delete" Problem

- If a row is physically deleted in the source, it has no "updated" timestamp and disappears from the input. The incremental loader will miss this change.
- **Mitigation:** Require producers to use "Soft Deletes" (e.g., `is_deleted=true` flag) so the change propagates via `UPDATE` instead of `DELETE`.

❖ Backfilling Risks

- Running a delta-column job over a long historical period (backfill) effectively becomes a full load, potentially crashing the pipeline due to resource limits.
- **Mitigation:** Slice the backfill into smaller time windows (e.g., process 1 hour at a time) to control resource usage.

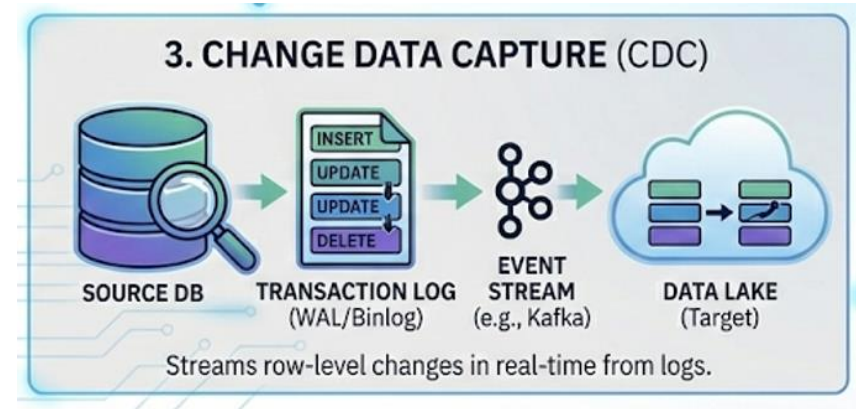
Pattern 3: Change Data Capture (CDC)

❖ The Low-Latency Alternative

- Used when batch incremental loading is too slow for business requirements or latency is critical.
- Captures row-level changes (Insert, Update, Delete) directly from the database transaction log (WAL, Binlog).

❖ Advantages

- **Completeness:** Captures "Hard Deletes" that query-based incremental loaders miss.
- **History:** Provides a stream of all intermediate states, not just the final state.



Pattern 4: Passthrough Replicator

❖ Scenario

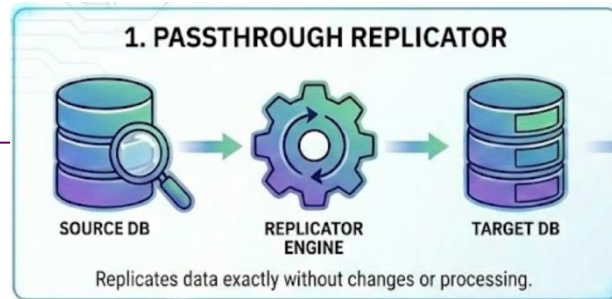
- Need to copy data "as-is" from Production to Staging/Dev environments for testing or redundancy.
- Critical when the source API is non-idempotent (returns different data at different times).

❖ Design Principles

- **Keep it Simple:** Use raw file-copy commands or simple line-based readers to avoid accidental type conversion (e.g., floating-point rounding errors).
- **Push Semantics:** The Production environment should "push" data to Staging to isolate Production from Staging's instability.

❖ Drawbacks

- Serialisation side effects, like badly formatted dates
- Production resources isolation
- Cannot replicate data if it contains PII (Personally Identifiable Information) that cannot be accessed outside production.



Pattern 5: Transformation Replicator

❖ Solution: Replicate with Transformation

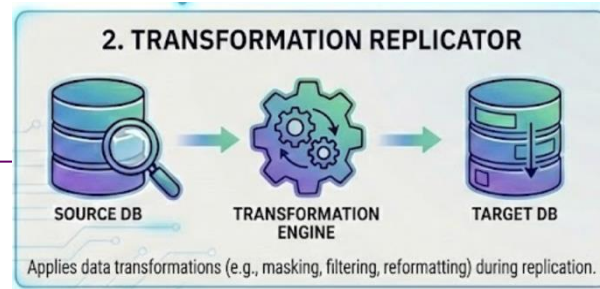
- Insert a lightweight transformation layer during the copy process to mask, hash, or drop sensitive columns.

❖ Implementation Techniques

- **Column Dropping:** Use `SELECT EXCEPT` or `drop()` to exclude sensitive fields entirely.
- **Data Masking:** Apply `substring` or hashing functions (e.g., `SUBSTRING(name, 1, 1)`) to preserve data utility while hiding actual values.

❖ Drawbacks

- Higher risk of misformatted attributes due to the schema-based transformation
- Mapping rules and PII data definition must be up to date



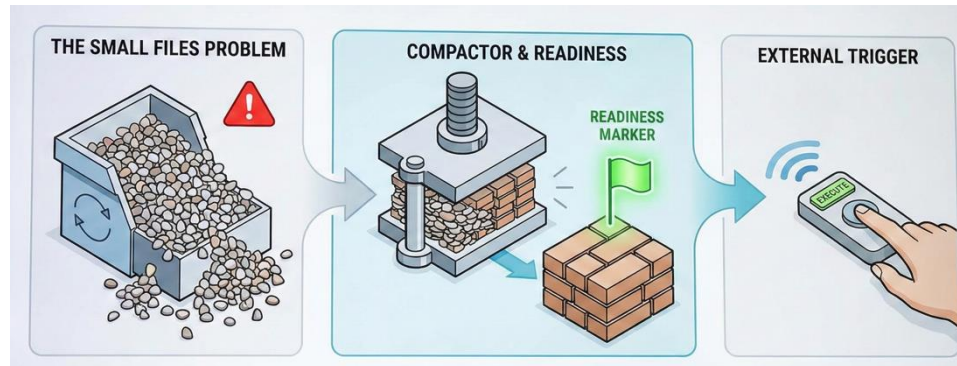
Technical Challenge: The Small Files Problem

❖ Problem

- Ingesting data frequently often results in thousands of tiny files (KB size).

❖ Impact

- **Metadata Overhead:** Engines spend more time listing files than reading data (70% listing / 30% reading).
- **Cost:** Object stores (e.g., S3) charge per API request; thousands of small PUT/GET requests inflate costs significantly.



Pattern 6: Compactor

❖ Solution to Small Files

- A background process that reads small files and rewrites them into larger, optimized files to reduce I/O overhead.

❖ Implementation Approaches

- **Table Formats:** Delta Lake OPTIMIZE or Iceberg rewrite data files automatically merge files while maintaining ACID consistency.
- **Kafka Compaction:** Retains only the latest value for a specific key, reducing storage growth over time.

❖ Housekeeping (VACUUM)

- Compaction creates new large files but doesn't immediately delete the old small ones (to support Time Travel).
- You must run a VACUUM job to physically delete obsolete files and reclaim storage space.

Pattern 7: Readiness Marker

❖ The Coordination Problem

- Downstream consumers (ML models, Dashboards) often try to read datasets before the ingestion is complete, leading to partial results.

❖ Solution: Signalling Completeness

- **Flag Files:** Write a `_SUCCESS` or `COMPLETED` file only after the data write operation finishes successfully.
- **Convention:** Consumers listen for the existence of the flag file before beginning their processing.

❖ Example

- **Manifest Files:** In Snowflake data loading, generating a JSON manifest file listing all successfully written objects. The consumer reads the manifest to know exactly which files constitute the "complete" batch, ensuring atomic ingestion.

Pattern 8: External Trigger

❖ Moving from Pull to Push

- **Pull (Polling):** A scheduled job checks for new data every 5 minutes.
 - Inefficient if data arrives sporadically (resource waste).
- **Push (Event-Driven):** The data producer sends a notification (Event) when data is ready, triggering the ingestion pipeline immediately.

❖ Example

- **Notification Handler:** A serverless function (e.g., AWS Lambda) subscribes to file creation events.
- **Action:** The handler triggers an Airflow DAG via API, passing context (filename, timestamp) as parameters.

Summary & Key Takeaways

❖ Ingestion is Non-Trivial

- It requires handling unpredictable volumes, hostile sources, and strict latency requirements.

❖ Pattern Selection Guide

- Use Full Loader for small reference data.
- Use Incremental Loader for growing logs/events.
- Use CDC for low-latency database replication.
- Use Compactor if your ingestion creates many small files.
- Use Readiness Markers to prevent downstream data corruption.

Error Management Design Patterns

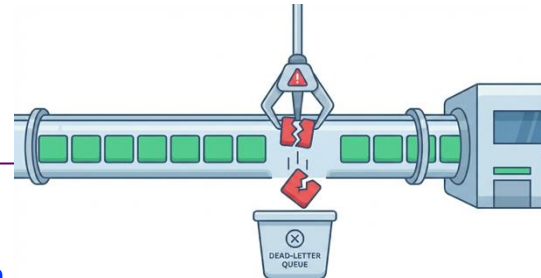


Error Management Design Patterns

- ❖ Context: The Reality of Data Engineering
 - "In this world, nothing can be said to be certain, except errors and data quality issues".
- ❖ Data engineers inherit issues from upstream producers
 - such as unreliable networks leading to late delivery or retries causing duplicates.
- ❖ **The Goal:**
 - Move from fragile "happy path" pipelines to robust systems that handle transient and non-transient errors gracefully.

Pattern 1: Dead-Letter

Managing Unprocessable Records



❖ The Problem

- A single "poison pill" record (e.g., corrupted JSON) can crash an entire batch or block a stream, halting data delivery for all downstream consumers.

❖ The Solution

- Instead of failing the job, catch the exception, sequester the bad record into a separate storage area (Dead-Letter Queue/Storage), and continue processing valid records.
- **Metadata Decoration:** Enrich the failed record with context (*exception message*, *timestamp*, *job ID*) to facilitate debugging.

❖ Example

- Schema Violation: A pipeline expects an age field to be an Integer. A source system sends "Twenty". The job catches the `TypeConversionException`, writes the payload to an "Bad Records" bucket, and alerts the data steward.

Dead-Letter Challenges

❖ The Snowball Backfilling Effect

- If you fix and replay the dead-lettered records later, you might modify partitions that downstream consumers have already processed.
- This forces downstream consumers to re-process (backfill) their data, potentially triggering a chain reaction of backfills across the entire platform.

❖ Ordering and Consistency

- Replaying failed events breaks time-series ordering. If event t_1 fails and is replayed after t_3 , the sequence becomes $t_2 \rightarrow t_3 \rightarrow t_1$.
- This invalidates logic relying on strict ordering, such as sessionization based on inactivity windows.

Pattern 2: Windowed Deduplicator

❖ The Problem

- Distributed systems rarely guarantee "Exactly-Once" delivery; "At-Least-Once" is the standard, meaning retries generate duplicates.

❖ The Solution

- Define a uniqueness key and a time window. The system stores the keys of seen records in a "State Store" and drops duplicates arriving within that window.
- **Batch Implementation:** Uses `dropDuplicates()` or `ROW_NUMBER() = 1` over the entire dataset.
- **Streaming Implementation:** Uses Watermarks to define how long to keep state. Old state is purged to prevent memory explosion.

❖ Example

- **Idempotency Token:** An API receiver caches a hash of the request payload in Redis with a 24-hour TTL. If the same payload arrives again, it is rejected immediately before hitting the expensive processing logic.

Pattern 3: Late Data Detector

❖ Event Time vs. Processing Time

- **Event Time:** When the action happened (user clicked).
Processing Time: When the pipeline received it.
- Network delays cause data to arrive late relative to its Event Time.

❖ Watermarks

- A mechanism to measure progress in Event Time.
Defined as $\text{MAX}(\text{event_time}) - \text{allowed_lateness}$.
- Records arriving with an event time older than the current Watermark are classified as "Late" and can be dropped or handled separately.

❖ Aggregation Pitfalls

- Using $\text{MIN}(\text{event_time})$ across partitions allows the slowest producer to hold back the entire system ("Stuck in the Past"). MAX is preferred for monotonicity.

Pattern 4: Static Late Data Integrator

❖ Scenario

- You need to include late data (e.g., offline mobile events) into your daily batches, but you cannot wait indefinitely.

❖ The Solution

- Define a fixed "Lookback Window" (e.g., 15 days). Every daily run processes the current day T plus reprocesses the past $T-15$ days to catch any late arrivals.

❖ Architecture Strategies

- **Sequential**: Integrate late data -> Process current execution. Essential for stateful pipelines where today depends on yesterday.
- **Parallel**: Process current execution and late data integration simultaneously. Good for stateless inputs.

Pattern 5: Dynamic Late Data Integrator

❖ The Problem

- A static 15-day window is wasteful (reprocessing days with no changes) and insufficient (misses data arriving 16 days late).

❖ The Solution

- Track Last Update Time **vs.** Last Processed Time for every partition in a State Table.
- Dynamically query this table to identify only the specific partitions that have received new data since the last run.

❖ Implementation Challenge

- Requires getting "Last Modified" timestamps from the storage layer (e.g., Delta Lake Change Logs or BigQuery Partition Metadata).

Pattern 6: Filter Interceptor

❖ The Problem

- A sudden drop in data volume might be due to a bug in a filtering condition (e.g., `type IS NOT NULL`), not actual data changes.
- Execution plans often collapse filters, making it impossible to see which specific condition is removing rows.

❖ The Solution

- Wrap filters with accumulators or counters. Instead of silently dropping rows, increment a metric for every record filtered out by a specific rule.
- **SQL Implementation:** Use a `CASE WHEN` statement to flag invalid rows (e.g., `is_null_type`) before filtering, then aggregate counts by flag.

Pattern 7: Checkpointer (Fault Tolerance)

❖ The Necessity of State

- Streaming jobs are long-running and stateful (e.g., counting visits over 10 minutes). If a job crashes, in-memory state is lost.

❖ The Solution

- Periodically persist the processing position (offsets) and internal state to durable external storage.
- **Framework Managed:** Spark/Flink write state to a checkpoint location automatically.
- **Manual:** A consumer manually commits offsets to Kafka (`_consumer_offsets`) or DynamoDB after processing a batch.

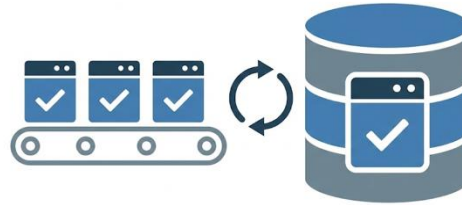
❖ Trade-off

- Latency vs. Guarantee: Frequent checkpoints minimize data replayed after a crash but add latency overhead to the happy path.

Summary

- ❖ Resilience is a Feature, Not an Afterthought
- ❖ Unprocessable Data
 - Don't crash; use Dead-Letter queues.
- ❖ Duplicates
 - Don't count twice; use Windowed Deduplicators.
- ❖ Timing
 - Don't ignore reality; use Late Data Detectors and Integrators to handle network chaos.
- ❖ Observability
 - Don't fly blind; use Filter Interceptors to track data loss.
- ❖ Recovery
 - Don't start from zero; use Checkpointers to save progress.

Idempotency Design Patterns



Idempotency Design Patterns

- ❖ **Context:** The Mathematical Guarantee of Consistency
 - In data engineering, errors are inevitable, and retries are the standard recovery mechanism.
- ❖ **The Challenge:** A retried task might execute a write operation twice. If the operation is not idempotent, this leads to duplicates or corrupted data.
- ❖ **Definition:** An idempotent operation produces the same result regardless of how many times it is executed.
 - Math Analogy: The absolute function is idempotent: $\text{abs}(-1) == \text{abs}(\text{abs}(-1))$.
 - Data Analogy: Running a daily ingestion job three times should result in only one day's worth of data in the target table.

Pattern 1: Fast Metadata Cleaner

❖ The Problem

- Removing data physically (`DELETE`) is slow on large datasets (TB scale) because it requires scanning files to identify rows.

❖ The Solution

- Leverage metadata operations (`TRUNCATE TABLE`, `DROP TABLE`, `DROP PARTITION`) which act on the logical layer rather than the physical layer.
- **Implementation:** Organize data into granular partitions (e.g., Weekly or Daily tables).
 - **Analyze** Context: Determine which partition is being processed.
 - **Clean:** Drop or Truncate that specific partition/table.
 - **Write:** Load the new data.

❖ Example

- **Scenario:** A generic "Visits" dataset is physically stored as 52 separate tables (`visits_week_1`, `visits_week_2...`).
- **Execution:** When reprocessing Week 1, the job drops `visits_week_1` instantaneously and recreates it, ensuring zero duplicates without expensive row-level deletes.

Fast Metadata Cleaner Challenges

❖ Granularity and Backfilling

- The pattern binds your backfilling capability to your partition size.
- If you need to fix data for just one user in a weekly partition, you must truncate and reprocess the entire week.

❖ Quota Limits

- Cloud Data Warehouses often limit the number of tables or partitions (e.g., BigQuery limits partitions to 4,000).
- Creating a new table for every hour of data will hit these limits quickly.

❖ Schema Evolution

- If you add a column, you must update the schema for all existing partitions, or the "Union View" that exposes them will fail.

Pattern 2: Data Overwrite

❖ The Problem

- You are working with an Object Store (S3, GCS) or a system that does not support metadata commands like `TRUNCATE`.

❖ The Solution

- Use the `INSERT OVERWRITE` semantic.
 - This command implicitly deletes the target data before writing the new dataset in a single atomic action.
- Mechanism:
 - **SQL**: `INSERT OVERWRITE INTO target SELECT * FROM source`
 - **Spark**: `df.write.mode('overwrite').save(...)`

❖ Example

- *Scenario*: A daily batch job calculates "Daily Active Users".
- *Action*: The job writes to `s3://bucket/data/date=2023-10-27/`. Even if the job runs 5 times, Spark deletes the contents of that specific directory before writing the file, guaranteeing the directory contains only the result of the last successful run.

Pattern 3: Merger (Upsert)

❖ The Problem

- You cannot overwrite the whole table because you are processing an incremental feed (only new/changed records). Overwriting would delete historical data.

❖ The Solution

- Use the `MERGE` (or `Upsert`) statement to combine new data with existing data based on a unique key.
- Logic:
 - **MATCHED**: Update the existing record.
 - **NOT MATCHED**: Insert the new record.
 - **MATCHED + DELETED Flag**: Delete the record (Soft Delete handling).

❖ Example

- *Scenario*: Syncing a Users database to a Data Lake.
- *Action*: The source sends a stream of changes. The pipeline runs a `MERGE on user_id`. If `user_id=101` exists, update the email; if not, insert the new user.

Pattern 4: Stateful Merger

❖ The Backfilling Inconsistency

- The standard Merger pattern is stateless. If you backfill a job from 3 days ago, you might merge "old" data on top of "new" data, corrupting the current state.

❖ The Solution

- Introduce a **State Table** that tracks the version of the target table corresponding to each pipeline execution time.
- Workflow:
 - **Check State**: Before merging, check if this execution time has run before.
 - **Restore**: If it has run, use Time Travel (Delta Lake/Iceberg) to restore the target table to its previous state before the merge.
 - **Merge**: Proceed with the merge operation.

❖ Trade-off

- Requires a storage engine that supports Versioning/Time Travel. Without it, you must manually backup/restore raw tables, which is expensive.

Pattern 5: Keyed Idempotency

❖ The Problem

- You are using a Key-Value store (e.g., Cassandra, DynamoDB) where `MERGE` operations are not available or too slow.

❖ The Solution

- Generate a deterministic Unique Key based on the data content itself. If the same data is processed twice, it generates the same key and overwrites the existing entry idempotently.

❖ The "Late Data" Trap

- Do not use mutable attributes (like "Processing Time") to generate the key.
- Bad Key: `hash(user_id + current_timestamp)`. A retry will generate a new key and create a duplicate.
- Good Key: `hash(user_id + event_timestamp)`. A retry will generate the same key and overwrite the previous entry.

Pattern 6: Transactional Writer

❖ The Problem

- A distributed job (like Spark) runs multiple tasks in parallel. If one task fails and retries, it might write partial data, leading to duplicates or "dirty reads" for consumers.

❖ The Solution

- Wrap the entire write operation in an ACID transaction.
- **All-or-Nothing**: Data is written to a temporary stage or hidden state. It is only made visible (Committed) when all tasks complete successfully.

❖ Implementation

- **Databases**: BEGIN TRANSACTION -> INSERT -> COMMIT.
- **File Formats**: Delta Lake/Iceberg use a transaction log (`_delta_log`). Files are written but ignored by readers until an entry is added to the log.

Pattern 7: Proxy (Immutable Datasets)

❖ The Problem

- Legal or compliance requirements (audit trails) forbid overwriting or deleting data. You must keep every version ever generated.

❖ The Solution

- Never overwrite. Always write to a new, timestamped table (e.g., `sales_20231027_v1`, `sales_20231027_v2`).
- **The Proxy**: Use a View or a Manifest File to point consumers to the latest valid version.
- **Switching**: When a new version is ready, update the View definition `CREATE OR REPLACE VIEW sales AS SELECT * FROM sales_20231027_v2`.

❖ Example - Blue/Green Deployment:

- Current View points to `Table_A`. Pipeline loads data into `Table_B`.
- Once `Table_B` is verified, the View is updated to point to `Table_B`.
- Consumers experience an atomic switch with no downtime or partial data.

Summary – Selecting the Right Pattern

- ❖ Full Dataset Replacement:
 - Use **Fast Metadata Cleaner** (if metadata ops supported) or Data Overwrite (if not).
- ❖ Incremental Updates:
 - Use **Merger** for standard cases, or **Stateful Merger** if backfilling consistency is critical.
- ❖ Key-Value Stores:
 - Use **Keyed Idempotency** with deterministic key generation.
- ❖ Strict Compliance:
 - Use **Proxy** to maintain immutable history while exposing a single current version.

Data Value Design Patterns



Data Value Design Patterns

❖ Context: Turning Raw Data into Assets

- Raw data sitting in storage is not yet a valuable asset. It lacks context, correlation, and structure.

❖ The Challenge:

- Raw events (e.g., website visits) are often isolated. To answer questions like "What do visitors using Chrome have in common?", we need to correlate distinct items.

❖ The Solution:

- Apply Data Value patterns to augment datasets via enrichment, decoration, aggregation, and ordering.

Pattern 1: Static Joiner (Data Enrichment)

❖ The Problem

- Raw events (e.g., `visit_event`) contain IDs (e.g., `user_id`) but lack context (e.g., `user_email`, `registration_date`).
- Analysis requires bringing reference data to the activity stream.

❖ The Solution

- Combine the dynamic dataset (events) with a static or slowly changing reference dataset using a JOIN operation.
- Implementation:
 - **SQL JOIN**: Standard relational approach.
 - **Materialized API**: Fetch data from an API, save it as a table, and then join.

❖ Handling Time (Slowly Changing Dimensions)

- **SCD Type 2**: History is tracked with `start_date` and `end_date` columns.
 - *Query*: Join on id AND event time BETWEEN `start_date` AND `end_date`.
- **SCD Type 4**: History is kept in a separate table.
 - *Current Table*: Stores only the latest state.
 - *History Table*: Stores all past versions.

Pattern 2: Dynamic Joiner (Streaming Enrichment)

❖ The Problem

- In streaming contexts, both the "fact" stream (visits) and the "dimension" stream (users) are moving.
- A Static Joiner fails here because the reference data might arrive later than the event data due to network latency.

❖ The Solution

- Use a stateful join with **Buffering**. Both streams buffer records for a specific time window to allow the slower stream to catch up.
- **Garbage Collection (GC) Watermark**: A mechanism to clean up old buffered state. If a record arrives after the watermark, it is dropped to preserve memory resources.

❖ Example

- *Scenario*: Joining AdClicks (Stream A) with AdImpressions (Stream B).
- *Action*: Stream A buffers clicks for 10 minutes. When the corresponding Impression arrives on Stream B, the join triggers. If the Impression arrives 20 minutes late, the buffer has been cleared, and the join is missed.

Pattern 3: Wrapper (Data Decoration)

❖ The Problem

- You need to add computed fields (e.g., `is_connected`) or standardized formats to raw data without losing the original raw structure for debugging.

❖ The Solution

- Wrap the original record inside a high-level envelope structure. Separate "Raw" data from "Computed" data.

❖ Implementation Strategies

- **Nested Structs:** `{"raw": { ...original... }, "computed": { ...new... } }`
- **Flat Map:** All columns at the root level (harder to distinguish).
- **Separated Tables:** Normalized approach storing raw and computed data in linked tables.

Pattern 4: Metadata Decorator

❖ The Problem

- You need to track technical context (Job Version, Ingestion Time, Batch ID) for observability, but exposing these fields to business users is confusing.

❖ The Solution

- Store technical context in a hidden metadata layer, separate from the business schema.
- Techniques:
 - **Kafka Headers:** Use record headers for metadata, keeping the payload clean.
 - **Hidden Columns:** Use columns that are excluded from public Views.
 - **Shadow Tables:** Store metadata in a separate table linked by ID, accessible only to engineers.

Pattern 5: Distributed Aggregator

❖ The Problem

- You need to compute statistics (Count, Sum, Average) on datasets that are too large to fit in the memory of a single machine.

❖ The Solution

- Leverage a distributed cluster (Spark, Flink, Hadoop).
- **Mechanism** (MapReduce):
 - **Map**: Process records locally.
 - **Shuffle**: Exchange records over the network so all records with the same Key end up on the same Node.
 - **Reduce**: Perform the final aggregation on the co-located records.

❖ Challenge: Data Skew

- If one key (e.g., "Unknown User") has millions of records, one node will be overwhelmed while others sit idle.
 - Mitigation: **Salting**. Add a random suffix to the key to distribute it, aggregate, then remove the salt and re-aggregate.

Pattern 6: Local Aggregator

❖ The Problem

- Shuffling data across the network is expensive (latency + bandwidth).
- If the data is already partitioned by the grouping key in the source (e.g., Kafka topic partitioned by `user_id`), the shuffle is redundant.

❖ The Solution

- Perform aggregations strictly within the processing node boundaries.
- **Prerequisite:** The producer must guarantee that all records for a specific key arrive at the same physical partition.
- **Benefit:** Tasks run in total isolation, significantly increasing throughput and reducing latency.

Pattern 7: Incremental Sessionizer (Batch)

❖ The Problem

- Users generate events over time. You want to group these into "Sessions" based on inactivity periods (e.g., 30 mins).
- In batch processing, a session might span across two hourly partitions (e.g., 09:50 to 10:10). Processing partitions in isolation breaks the session.

❖ The Solution

- Use three storage spaces:
 - **Input:** Raw new events.
 - **Pending Sessions:** Sessions that are open/active from the previous batch run.
 - **Completed Sessions:** Sessions that have closed.
- **Workflow:** Load Pending Sessions + New Input -> Merge -> Check Inactivity -> Write Closed Sessions to Output / Write Active Sessions back to Pending.

Pattern 8: Stateful Sessionizer (Streaming)

❖ The Problem

- Batch sessionization has high latency (e.g., 1 hour). Real-time dashboards need sessions updated in seconds.

❖ The Solution

- Use a stateful streaming job with an in-memory State Store (backed by a resilient log).
- Mechanism:
 - **Session Windows**: Automatically group events by key within a fixed gap duration.
 - **Arbitrary Stateful Processing**: Custom logic (e.g., mapGroupsWithState in Spark) to handle complex timeouts and state updates.

❖ Trade-off

- Requires managing **State Rebalancing** during scaling and handling **At-Least-Once** semantics (potential duplicates on restart).

Pattern 9: Bin Pack Orderer

❖ The Problem

- Delivering data to an API/Stream while maintaining strict order (e.g., Event Time).
- **Partial Commits**: Bulk APIs often succeed partially (Record A writes, Record B fails). Retrying B breaks the order A -> B.

❖ The Solution

- **Bin Packing**:
 - **Sort** records by Key and Time.
 - **Pack** into bins such that each Key appears only once per bin.
 - **Flush** bins sequentially.
- This allows using efficient Bulk APIs. Even if a bin fails and retries, the internal order of keys remains valid because no key is repeated within a single transmission unit.

Pattern 10: FIFO Orderer

❖ The Problem

- You need strict First-In-First-Out delivery for a simple use case without the complexity of bin packing.

❖ The Solution

- Sacrifice throughput for simplicity.
- **Mechanism:**
 - Concurrency = 1.
 - Send one record -> Wait for Ack -> Send next record.
- **Optimization:** Use "In-Flight Requests = 1" or Idempotent Producers (e.g., Kafka) to allow limited concurrency without breaking order.

Summary – Creating Value from Chaos

❖ **Enrichment:**

- Use **Static Joiner** for reference data, **Dynamic Joiner** for streaming correlation.

❖ **Decoration:**

- Use **Wrapper** to preserve raw data, **Metadata Decorator** to hide technical context.

❖ **Aggregation:**

- Use **Distributed Aggregator** for scale, **Local Aggregator** for speed (if partitioned).

❖ **Sessionization:**

- Use **Incremental** for batch efficiency, **Stateful** for real-time insights.

❖ **Ordering:**

- Use **Bin Pack Orderer** for bulk performance, **FIFO Orderer** for strict simplicity.

Data Flow Design Patterns



Data Flow Design Patterns

- ❖ **Context: Beyond Local Insight**
 - Generating value locally is a good start, but the real potential lies in sharing data assets across the organization (e.g., Marketing using Sales data).
- ❖ **The Challenge:** How do we coordinate this exchange? How do we sequence dependencies, merge parallel efforts, and orchestrate complex workflows efficiently?
- ❖ **The Solution:** Data Flow Patterns orchestrate the "Plumbing" of data engineering—sequencing, branching, and merging pipelines.

Pattern 1: Local Sequencer

❖ The Problem

- A single monolithic job becomes unmaintainable. If step 10 fails, you must restart from step 1, re-executing expensive operations.

❖ The Solution

- Decouple a big component into smaller, connected steps running sequentially within the same context.
- **Strategy:**
 - **Orchestration Layer:** Airflow tasks linked via `>>`. (e.g., Sensor `>>` Loader `>>` Transformer).
 - **Processing Layer:** Chaining functions in Spark code.

❖ Boundary Rules

- **Restart Boundaries:** If task B fails, should task A run again? If no, separate them. Ideally, place boundaries around expensive computations to save costs on retries.

Pattern 2: Isolated Sequencer

❖ The Problem

- Two different teams (e.g., Data Engineering and Data Science) manage separate pipelines. Merging them into one DAG is politically or technically impossible.

❖ The Solution

- Connect physically isolated pipelines via external triggers or data sensing.

❖ Implementation Strategies

- **Data-Based**: The Consumer pipeline uses a `Sensor` to wait for a "Success Marker" file generated by the Producer. (Loose Coupling).
- **Task-Based**: The Producer pipeline explicitly triggers the Consumer pipeline via an API call (e.g., `TriggerDagRunOperator`). (Tight Coupling).

❖ Trade-off

- **Tight Coupling** ensures immediacy but breaks if the Consumer renames their DAG. **Loose Coupling** is more robust but might incur polling latency.

Pattern 3: Aligned Fan-In

❖ The Problem

- You have partitioned data (e.g., 24 hourly files) and need to generate a single daily aggregate. Processing all 24 hours in one serial job is too slow; managing 24 separate pipelines is messy.

❖ The Solution

- Run 24 parallel "branches" (one for each hour). Once all branches succeed, a final "Merge" task consolidates the results.
- **Benefit:** Parallelism speeds up processing. If Hour 23 fails, you only rerun that specific branch, not the entire day.

❖ Example

- Scenario: Hourly Visits logs.
- Action: 24 parallel Spark jobs clean the hourly logs. The final step runs a `UNION` or Aggregation on the 24 clean datasets to produce the "Daily Summary."

Pattern 4: Unaligned Fan-In

❖ The Problem

- In the Aligned Fan-In, if one branch fails (e.g., Hour 23 is missing), the final aggregation is blocked. Sometimes, delivering a "Partial Result" (96% complete) is better than delivering nothing.

❖ The Solution

- Relax the dependency rule. The downstream task runs even if some parents fail.
- **Mechanism:**
 - **Trigger Rule:** Set to ALL_DONE instead of ALL_SUCCESS.
 - **Metadata:** Flag the output dataset as is_approximate=true or completeness=0.96 so consumers know it's partial.

❖ Use Case

- Real-time Dashboards: Displaying "Sales so far" is valuable even if one region's data is delayed.

Pattern 5: Parallel Split (Fan-Out)

❖ The Problem

- A single dataset needs to be processed for two different purposes (e.g., one for a Legacy system, one for a New system during migration).

❖ The Solution

- Read the source data once, then split the execution flow into two independent, parallel branches.

❖ Optimization (Caching)

- Since both branches share the same input, reading it twice is wasteful.
 - **Cache:** Use `df.persist()` in Spark to load raw data into memory once.
 - **Execute:** Both branches read from the memory cache, preventing double I/O on the source storage.

Pattern 6: Exclusive Choice

❖ The Problem

- A pipeline needs to adapt its behavior based on context (e.g., "If it's Monday, run the Weekly Report; otherwise, skip").

❖ The Solution

- Insert a "Router" or "Branching" task. This task evaluates a condition and determines which downstream path to execute, skipping the others.

❖ Implementation

- **Airflow:** `BranchPythonOperator` returns the `task_id` of the next task to run.
 - *Example:* if `execution_date >= '2024-01-01'`: return 'new_logic' else: return 'legacy_logic'.

❖ Best Practice

- Avoid complex nested if-else logic inside the data processing code ("Hidden Logic"). Lifting this logic to the orchestration layer makes the flow visible and debuggable.

Pattern 7: Single Runner (Orchestration)

❖ The Problem

- Incremental pipelines (e.g., Sessionization) often depend on the state of the previous run. Running two instances in parallel (e.g., Hour 1 and Hour 2) causes race conditions and data corruption.

❖ The Solution

- Enforce strict sequential execution. Configure the orchestrator to allow `max_active_runs = 1`.
- **Backfilling Consequence:** If you need to backfill 30 days, they will run one by one (30 x 1 hour = 30 hours). You cannot parallelize this without breaking the logic.

Pattern 8: Concurrent Runner (Orchestration)

❖ The Problem

- Pipelines processing independent data (e.g., loading independent files) are needlessly blocked by the "Single Runner" constraint, causing latency during backfills.

❖ The Solution

- Relax the concurrency limit. Allow `max_active_runs > 1`.
- **Benefit:** Backfilling 30 days can happen in parallel (limited only by cluster capacity). 30 days might finish in 1 hour if you have enough nodes.

❖ Risk: Resource Starvation

- If you launch 100 concurrent runs, you might consume all cluster resources, blocking other critical production jobs. Use **Pools** or **Queues** to cap total resource usage.

Summary – Orchestrating the Flow

❖ Sequence

- Use **Local Sequencer** for simple chains, **Isolated Sequencer** for cross-team dependencies.

❖ Fan-In

- Use **Aligned Fan-In** for strict completeness, **Unaligned Fan-In** for resilience/partial availability.

❖ Fan-Out

- Use **Parallel Split** for simultaneous processing, **Exclusive Choice** for conditional logic.

❖ Concurrency

- Use **Single Runner** for stateful/incremental logic, **Concurrent Runner** for independent/stateless workloads.

Data Security Design Patterns



Data Security Design Patterns

❖ Context: Data as a Liability

- Valuable data assets are targets for malicious actors and subject to strict regulations (GDPR, CCPA).

❖ The Challenge:

- How do we balance accessibility for business value with the need for privacy, compliance (Right to be Forgotten), and access control?

❖ The Solution:

- Implement patterns for efficient data removal, fine-grained access, encryption, and secure connectivity.

Pattern 1: Vertical Partitioner (Data Removal)

❖ The Problem

- "Right to be Forgotten" requests require deleting a user's personal data. If `user_id` and `email` are repeated in every row of a billion-row event log, deleting one user requires rewriting terabytes of data.

❖ The Solution

- Split the dataset into two vertical partitions:
- **Mutable/PII Data**: Stores sensitive fields (Email, Address) linked to an ID. Stored in a format easy to update/delete.
- **Immutable/Event Data**: Stores non-sensitive event details (Click, PageView) linked to the same ID.

❖ Benefit

- When a deletion request arrives, you only delete the single row in the PII table. The billion event rows remain but are now effectively anonymized because the ID can no longer be resolved to a person.

Pattern 2: In-Place Overwriter



❖ The Problem

- You have a legacy dataset where PII is mixed with event data, and you cannot re-architect it into vertical partitions. You still need to comply with deletion requests.

❖ The Solution

- Rewrite the affected partitions or files to physically remove the specific records.

– Mechanism:

- **Native:** Use DELETE FROM table WHERE user_id = X (if supported, e.g., Delta Lake, Iceberg).
- **Simulation:** Read the file -> Filter out the user -> Write to Staging -> Atomic Swap/Rename to replace the original file.

❖ Challenge

- **I/O Heavy:** Deleting one record might require rewriting a 1GB file.
- **Vacuuming:** You must run a vacuum/compaction process to ensure the old "deleted" physical files are actually removed from disk.

Pattern 3: Fine-Grained Accessor (Tables)

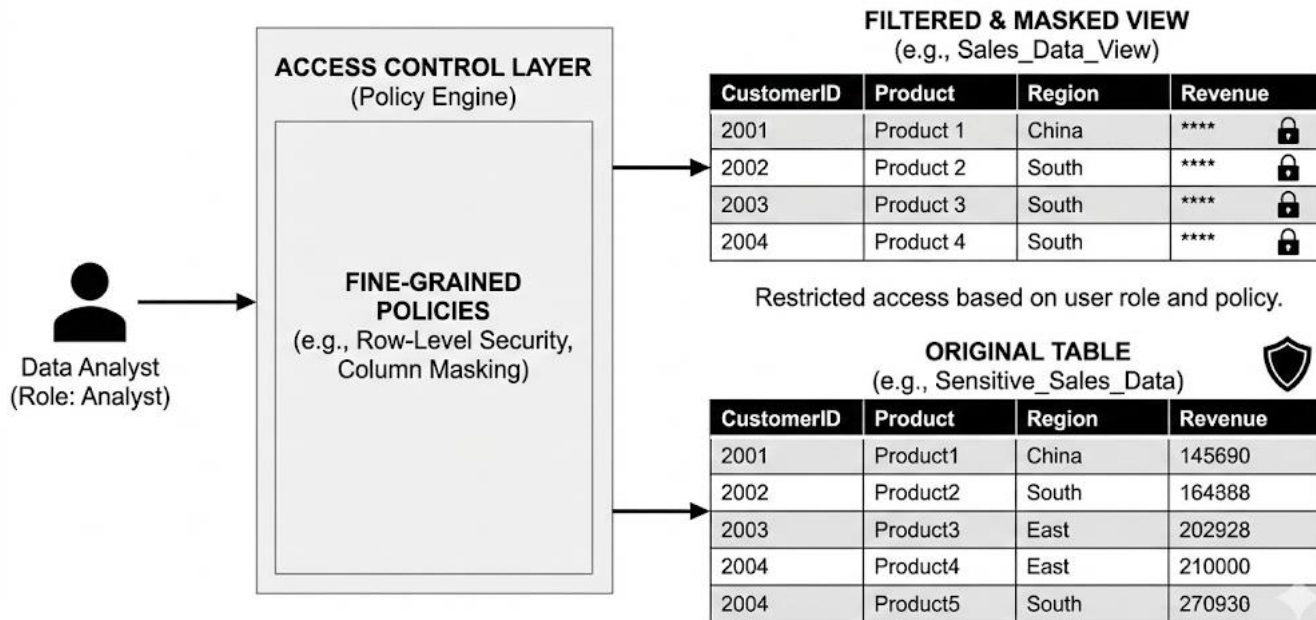
❖ The Problem

- Giving a user access to a table usually means they can see everything. But some users (e.g., Regional Managers) should only see specific rows (their region) or columns (non-sensitive metrics).

❖ The Solution

- Implement controls at the Column and Row level.
- **Column-Level:** `GRANT SELECT(col_a, col_b)`. The user gets an error if they try `SELECT *`. Or use Dynamic Masking to show `***` instead of values.
- **Row-Level (RLS):** Attach a policy function to the table. The database automatically appends a `WHERE` clause to every query (e.g., `WHERE region = current_user_region`).

Pattern 3: Fine-Grained Accessor (Tables)



Pattern 4: Fine-Grained Accessor (Resources)

❖ The Problem

- Data pipelines often run with broad permissions ("Admin" or "Read/Write All Buckets"), violating the Principle of Least Privilege. If a job is compromised, all data is at risk.

❖ The Solution

- Restrict access at the Infrastructure level (Cloud IAM).
- **Identity-Based**: Attach a policy to the Job's Service Account: "Can only read Bucket A and write to Bucket B."
- **Resource-Based**: Attach a policy to the Bucket itself: "Only the 'Marketing-ETL' role can read this bucket."

Pattern 5: Encryptor

❖ The Problem

- Physical theft of disks or interception of network traffic could expose data. Access Control Lists (ACLs) logic might be bypassed if someone accesses the raw storage directly.

❖ The Solution

- **At Rest:** Server-Side Encryption (SSE). The storage service encrypts data on disk using a key from a Key Management Service (KMS). Access requires both file permissions and key decryption permissions.
- **In Transit:** Enforce TLS 1.2+ for all connections.

Pattern 6: Anonymizer

❖ The Problem

- You need to share data with a third party (e.g., for analytics) but cannot share PII due to user consent or regulation.

❖ The Solution

- Irreversibly transform the dataset before sharing.
- Techniques:
- **Removal:** Simply drop the `birth_date` column.
- **Perturbation:** Add random noise to values (e.g., shift location coordinates slightly).
- **Synthetic Replacement:** Replace real emails with fake ones generated by a model (Faker), preserving the format but removing the reality.

Pattern 7: Pseudo-Anonymizer

❖ The Problem

- Total anonymization destroys analytic value. You need to join datasets or track unique users over time without revealing their identity.

❖ The Solution

- Replace PII with a consistent token or hash.
- **Tokenization**: Swap Alice for Token_123. Store the mapping in a secure Vault. Reversible only by Vault admins.
- **Hashing**: One-way hash (SHA-256). Alice always becomes a1b2.... Allows joining two datasets on User ID without seeing the name.

❖ Risk

- **Re-identification**: Combining pseudo-anonymized data with external public data might reveal identities (e.g., "The only person in Zip Code X with Job Title Y").

Pattern 8: Secrets Pointer

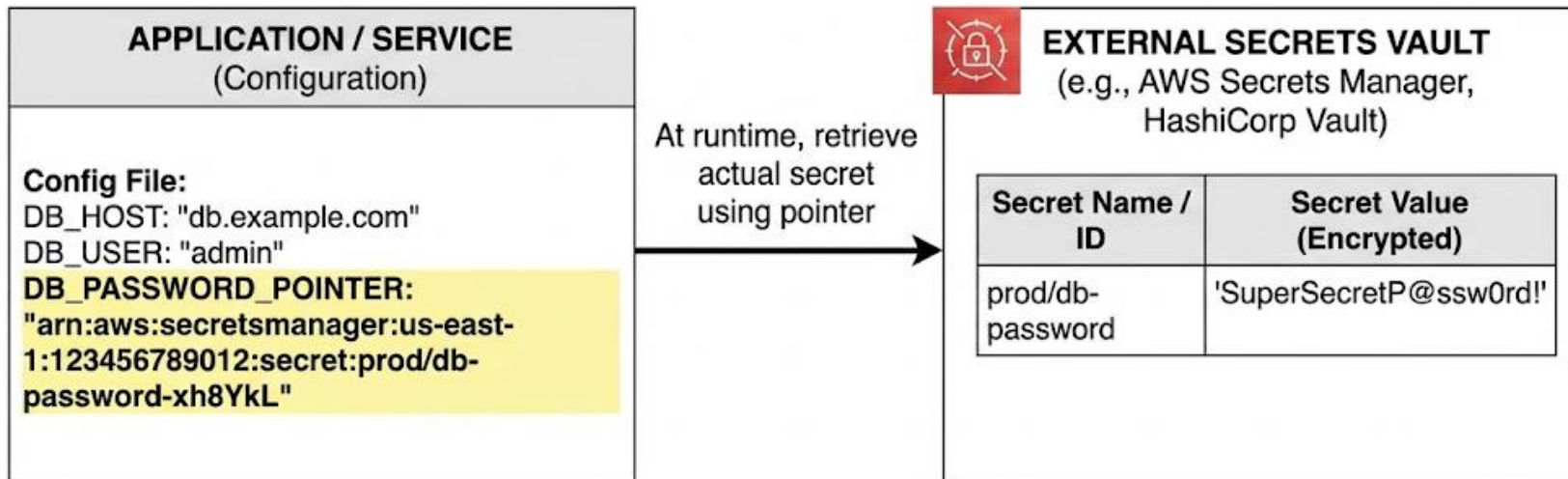
❖ The Problem

- Hardcoding database passwords or API keys in code/Git is a massive security risk.

❖ The Solution

- Store the actual credential in a Secrets Manager (Vault, AWS Secrets Manager).
- In the code, store only the pointer (the name/ID of the secret). At runtime, the application calls the Secrets Manager to retrieve the actual password.
- **Benefit:** Credentials can be rotated centrally without changing application code.

Pattern 8: Secrets Pointer



Actual sensitive data is stored securely and referenced, not hardcoded.

Pattern 9: Secretless Connector

❖ The Problem

- Even with Secrets Pointers, the application still handles a password (in memory). Long-lived credentials (like API Keys) can be leaked.

❖ The Solution

- Use Identity-Based Authentication (Workload Identity).
- **Mechanism:** The infrastructure (e.g., Kubernetes, AWS EC2) provides a temporary, auto-rotating identity token to the running job. The target service (Database, S3) trusts this identity directly.
- **Result:** No passwords exist. Authentication is handled via short-lived tokens and IAM trust relationships.

Summary – Security Layers

❖ Removal

- Use **Vertical Partitioner** for efficient deletion, **In-Place Overwriter** for legacy cleanup.

❖ Access

- Use **Fine-Grained Accessors** for Tables (SQL) and Resources (IAM).

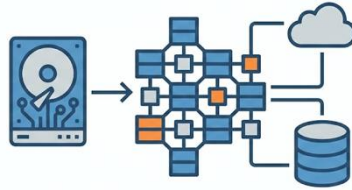
❖ Protection

- Use **Encryptor** for physical security, **Anonymizer** for external sharing.

❖ Connectivity

- Use **Secrets Pointer** to hide passwords, **Secretless Connector** to eliminate them entirely.

Data Storage Design Patterns



Data Storage Design Patterns

- ❖ Context: The Time Factor in Data Engineering
 - Query latency is a critical metric. Long-running queries increase costs (compute time) and decrease business agility.
- ❖ Optimization Strategies:
 - **Reactive**: Add more compute resources (Vertical/Horizontal Scaling). Fast but expensive.
 - **Preemptive**: Organize data intelligently on disk. Reduces I/O and CPU usage significantly.
- ❖ The Goal:
 - Skip irrelevant data. The fastest data to process is the data you never read.

Pattern 1: Horizontal Partitioner

❖ The Problem

- Analyzing a single day of data from a multi-year dataset requires scanning the entire table if the data is stored as a monolith.

❖ The Solution

- Divide the dataset into physically isolated distinct storage segments based on a specific attribute (e.g., `event_date`).
- Mechanism:
 - **Directory Structure:** `s3://bucket/data/year=2024/month=01/day=01/`.
 - **Pruning:** The query engine reads only the directories matching the WHERE clause, skipping years of irrelevant data.

❖ Challenges

- **Cardinality:** Using high-cardinality keys (e.g., `user_id`) creates millions of tiny partitions/files ("Small Files Problem"), overloading the metadata layer.
- **Skew:** If one partition is massive (e.g., Black Friday data) while others are small, processing becomes bottlenecked by the "Straggler" partition.

Pattern 2: Vertical Partitioner

❖ The Problem

- A table contains frequently accessed "Hot" columns (e.g., price, status) mixed with rarely accessed "Cold" columns (e.g., full_description, binary_blob).
- Row-oriented storage reads the entire row, wasting I/O on Cold columns.

❖ The Solution

- Split the logical table into multiple physical tables or files based on column groups.
- Application:
 - **Performance:** Store heavy text blobs separately from lightweight analytics columns.
 - **Lifecycle:** Store mutable attributes (e.g., status) separately from immutable attributes (e.g., created_at) to optimize updates.

❖ Trade-off

- Requires **Joins** to reconstruct the full record, which adds network overhead if the user needs both Hot and Cold data simultaneously.

Pattern 3: Bucket

❖ The Problem

- High-cardinality attributes (e.g., `user_id`) cannot be used for Horizontal Partitioning. However, you often `JOIN` or `GROUP BY` these columns.

❖ The Solution

- Group records into a fixed number of "Buckets" using a hash function: $\text{bucket} = \text{hash}(\text{key}) \% N$.
- Benefits:
 - **Join Optimization:** If Table A and Table B are bucketed by the same key into the same number of buckets, the join engine knows that `Key=123` is in Bucket 5 for both tables. It can join them locally without shuffling data across the network (Sort-Merge Join).
 - **Sampling:** You can easily sample 10% of users by reading 10% of the buckets.

Pattern 4: Sorter

❖ The Problem

- Even within a partition, data is randomly distributed. Finding `id=500` requires scanning the whole file.

❖ The Solution

- Physically sort the data on disk during the write process.
- **Lexicographical Sort**: Simple hierarchical sort (`ORDER BY City, Date`). Good for queries matching the prefix.
- **Z-Order (Curved Sort)**: Interleaves bits of multiple columns to map multidimensional data to 1D storage.
 - **Benefit**: Equally efficient for filtering on Column A, Column B, or Both. Standard sort heavily biases towards the first column.

Pattern 5: Metadata Enhancer

❖ The Problem

- Reading a file to check if it contains relevant data is expensive. You want to know before opening the file.

❖ The Solution

- Store statistics about the data in a lightweight metadata layer (e.g., File Footers, Commit Logs).
- **Mechanism (Min/Max Skipping):**
 - Parquet files store the min and max values for each column in the file footer.
 - Query: `SELECT * FROM table WHERE id = 500.`
 - Action: Engine checks metadata. File A range is [0-100]. Skip. File B range is [400-600]. Read.

❖ Implementation

- Modern table formats (Delta Lake, Iceberg) store these stats in the transaction log JSON/Avro files, allowing massive skipping without even listing the data files.

Pattern 6: Dataset Materializer

❖ The Problem

- Complex queries (aggregations, joins) are computationally expensive. Running them for every dashboard refresh wastes resources and increases latency.

❖ The Solution

- Pre-compute and save the result of the query to disk (Materialized View or Table).
- Trade-off:
 - **Read Speed:** Instant.
 - **Data Freshness:** Data is static until the next refresh.
 - **Write Cost:** Refreshing the view is expensive.
- **Incremental Refresh:** Advanced implementations (like Snowflake/BigQuery Materialized Views) only process the delta of new data to update the view, rather than recomputing from scratch.

Pattern 7: Manifest

❖ The Problem

- "Listing" files in an Object Store is slow and eventually consistent. For a table with 100,000 files, just finding the files takes longer than reading them.

❖ The Solution

- Maintain an explicit list of valid data files in a "Manifest" file.
- **Workflow:**
 - Writer creates data files.
 - Writer generates a Manifest (e.g., manifest.json) listing the paths of the new files.
 - Reader reads only the Manifest to locate data, bypassing the slow directory listing.
- Used extensively by Delta Lake, Iceberg, and Cloud Data Warehouses (Redshift/Snowflake) for consistent, fast reads.

Pattern 8: Normalizer (Data Representation)

❖ The Principle

- **Third Normal Form (3NF)**: Eliminate redundancy. Split data into Entity tables (Users, Products) and Relationship tables (Orders).
- **Snowflake Schema**: A dimensional model where dimensions are normalized into sub-dimensions (e.g., DateDim links to MonthDim links to QuarterDim).

❖ Pros & Cons

- **Pro**: Consistency. Updates are easy (update the User's name in one place, and it reflects everywhere).
- **Con**: Performance. Reading requires many expensive JOIN operations.

Pattern 9: Denormalizer (Data Representation)

❖ The Principle

- Optimize for Read performance by duplicating data.
- **Star Schema**: Dimensions are denormalized. The DateDim contains Month and Quarter directly. One level of join only.
- **One Big Table (OBT)**: Join everything into a single massive table. Orders table contains User_Name, Product_Price, Store_City.

❖ Pros & Cons

- **Pro**: Speed. Zero or few joins required for analysis.
- **Con**: Consistency anomalies. If a user changes their name, you must update millions of rows in the OBT. Storage costs are higher due to redundancy (mitigated by columnar compression).

Summary – Designing for Performance

❖ Organization:

- Use **Horizontal Partitioning** for low-cardinality time filters.
- Use **Bucketing** for high-cardinality join keys.
- Use **Sorting (Z-Order)** for multidimensional filtering.

❖ Optimization:

- Use **Metadata Enhancer** (Stats) to skip files.
- Use **Manifests** to skip listings.
- Use **Materializers** to skip computation.

❖ Representation:

- Use **Normalization** for write-heavy/consistent workloads.
- Use **Denormalization** for read-heavy/analytical workloads.

Data Quality Design Patterns



Data Quality Design Patterns

❖ Context: The Trust Deficit

- Data exchange is a mutual transaction based on trust. Poor quality (inaccuracy, incompleteness, inconsistency) destroys this trust.

❖ The Challenge:

- Pipelines often fail silently or propagate bad data downstream, leading to flawed business decisions (e.g., stopping a marketing campaign because "Unique Visitors" dropped by 50% due to a bug).

❖ The Solution:

- Implement patterns to **Enforce**, **Control** (Schema), and **Observe** data quality.

Pattern 1: Audit-Write-Audit-Publish (AWAP)

❖ The Problem

- Standard pipelines (Extract-Transform-Load) often load bad data first and discover issues later. Once data is loaded into production, the damage is done.

❖ The Solution

- Insert **explicit Audit** steps before and after transformation.
- Workflow:
 - **Input Audit:** Validate source file (e.g., file size > 0, JSON validity) before processing.
 - **Transform:** Run business logic.
 - **Output Audit:** Validate the transformed result (e.g., user_id is never NULL).
 - **Publish/Load:** Only if audits pass, write to the final production table.
- **Audit Failure Strategies:**
 - **Block:** Fail the pipeline immediately.
 - **Dispatch:** Route bad records to a "Dead Letter" storage and publish the rest.

Pattern 2: Constraints Enforcer

❖ The Problem

- Writing custom validation logic (AWAP) for every field is tedious and error-prone. Pipelines become cluttered with `if x is None: raise Error`.

❖ The Solution

- Delegate validation to the storage engine or format. Define rules declaratively.
- **Types of Constraints:**
 - **Type:** age must be an Integer.
 - **Nullability:** user_id cannot be NULL.
 - **Value:** event_time must be $< \text{NOW}() + 1 \text{ second}$ (Check Constraints).
 - **Integrity:** order.user_id must exist in the Users table (Foreign Keys).

❖ Implementation

- **Databases:** ALTER TABLE ADD CONSTRAINT CHECK (age > 0).
- **Formats:** Protocol Buffers (`protovalidate`) or Avro schemas enforce types at serialization time.

Pattern 3: Schema Compatibility Enforcer

❖ The Problem

- Producers change data structures (rename columns, change types) without notifying consumers, causing downstream pipelines to crash.

❖ The Solution

- Use a central **Schema Registry** or enforcement logic to reject incompatible changes at write time.
- Compatibility Modes:
 - **Backward**: New consumers can read old data (e.g., deleting a field is OK).
 - **Forward**: Old consumers can read new data (e.g., adding an optional field is OK).
 - **Full**: Both backward and forward guarantees.

❖ Example

- Kafka Schema Registry: If a producer tries to send a message missing a required field `visit_id`, the Registry rejects the write with a 409 Conflict error, protecting all consumers.

Pattern 4: Schema Migrator

❖ The Problem

- Strict enforcement prevents breaking changes, but sometimes you must make a breaking change (e.g., renaming `from_page` to `referral` for clarity).

❖ The Solution

- Perform a controlled migration over a "Grace Period."
- Workflow (Rename Example):
 - **Add** the new column `referral`. Populate it with the same data as `from_page`.
 - **Wait** (Grace Period) for consumers to update their queries to use `referral`.
 - **Delete** the old column `from_page` once lineage shows zero usage.
- Trade-off: Temporary storage increase (duplication) for the sake of stability.

Pattern 5: Offline Observer

❖ The Problem

- Rules and Constraints catch known errors. But what about "unknown unknowns" (e.g., data drift, shifting distributions)? Blocking the pipeline for these is too aggressive.

❖ The Solution

- Run a separate, non-blocking job to compute metrics on the processed data.
- Metrics:
 - Distribution of values (Mean, Max, Min).
 - Percentage of NULLs.
 - Row counts (Volume anomalies).
- Implementation: An Airflow DAG runs nightly, queries the production table, calculates stats, and alerts if $\text{today's_volume} < 0.5 * \text{yesterday's_volume}$.

Pattern 6: Online Observer

❖ The Problem

- Offline observation has latency. Detecting a "bad zip code format" regression 24 hours later means bad data has already been consumed by dashboards.

❖ The Solution

- Integrate observation directly into the generation pipeline.
- **Batch**: Calculate metrics immediately after the Transform step, before or parallel to Load.
- **Streaming**: Use accumulators or windowed aggregations to calculate "Lag" or "Invalid Record Counts" in real-time as data flows through the job.

❖ Trade-off

- Adds latency to the main pipeline. If the observation logic is heavy (e.g., profiling every column), it slows down production data delivery.

Summary – The Defense-in-Depth Strategy

❖ Enforce

- Use **AWAP** to block bad files and **Constraints Enforcer** to block bad rows.

❖ Control

- Use **Schema Compatibility Enforcer** to prevent structural breaks and **Schema Migrator** to manage evolution safely.

❖ Observe

- Use **Offline Observer** for deep, non-blocking analysis and **Online Observer** for critical, low-latency alerts.

Data Observability Design Patterns



Data Observability Design Patterns

❖ Context: Beyond Data Quality

- Data Quality patterns (like AWAP) validate the content of the data. However, they cannot detect when the system itself stops working silently or slows down.

❖ The Gap:

- If an upstream job fails to run, the AWAP check never triggers because there is no data to check.

❖ The Solution:

- Data Observability focuses on the health of the system via Detection (Data/Time) and Tracking (Lineage).

Pattern 1: Flow Interruption Detector

❖ The Problem

- "Silent Failure." A streaming producer crashes, or a batch file fails to land. The pipeline doesn't fail; it just processes zero records successfully. Downstream consumers starve.

❖ The Solution

- Monitor the "Heartbeat" of the data flow.
- Strategies:
 - **Continuous Delivery:** Alert if `records_per_minute == 0` for more than N minutes.
 - **Irregular Delivery:** Analyze time windows. If no data arrives for a duration exceeding the "Expected No-Data Window," trigger an alert.
- Implementation Layers:
 - **Metadata:** Check `last_modified_time` on the table.
 - **Storage:** Monitor file arrival timestamps in S3.

Pattern 2: Skew Detector (Volume Anomaly)

❖ The Problem

- A job runs successfully but processes only 50% of the expected volume (e.g., due to an upstream API failure dropping partial data).

❖ The Solution

- Compare current data volume against a historical baseline.
- Mechanism:
- **Comparison Window:** Compare Today vs. Yesterday or Last Week.
- **Threshold:** Set a tolerance (e.g., +/- 50%). If $\text{current_rows} < 0.5 * \text{previous_rows}$, fail or alert.

❖ Challenge

- **Seasonality:** Black Friday traffic will naturally skew compared to a normal Friday. Static thresholds create false positives; dynamic baselines (Moving Averages) are preferred.

Pattern 3: Lag Detector

❖ The Problem

- A streaming consumer is running without errors, but it is processing data slower than the producer is generating it. The system is drifting further away from real-time.

❖ The Solution

- Measure the distance between the "Head" of the stream (latest available) and the "Tail" (latest processed).
- Metrics:
 - Offset Lag: $\text{Latest_Offset} - \text{Committed_Offset}$.
 - Time Lag: $\text{Now}() - \text{Event_Time_of_Last_Processed_Record}$.
- **Aggregation:** Monitor the **Max Lag** (worst-case partition) or **P90 Lag** to understand overall system health.

Pattern 4: SLA Misses Detector

❖ The Problem

- A batch job scheduled for 06:00 usually finishes by 06:30. Today, it finishes at 08:30. Downstream dashboards were empty when the CEO looked at 08:00.

❖ The Solution

- Define and monitor Service Level Agreements (SLAs) for execution duration.
- Implementation:
 - **Batch**: $\text{Duration} = \text{End_Time} - \text{Start_Time}$. Alert if $\text{Duration} > \text{Max_Allowed}$.
 - **Streaming**: Measure end-to-end latency. Alert if $\text{Processing_Time} - \text{Event_Time} > 5$ minutes.

❖ Example

- **Airflow**: Define `sla=timedelta(minutes=30)` on a task. If the task runs longer, Airflow triggers a callback to page the on-call engineer.

Pattern 5: Dataset Tracker (High-Level Lineage)

❖ The Problem

- A dashboard is broken. You know the input table is corrupt, but you don't know which upstream team generates that table or which other dashboards are affected.

❖ The Solution

- Build a dependency graph linking Datasets (Nodes) and Jobs (Edges).
- **Automated:** Use tools like **OpenLineage** or **Marquez** that hook into Spark/Airflow to record inputs and outputs automatically at runtime.
- **Manual:** Parse SQL queries to extract `FROM` and `JOIN` tables to build a static map of dependencies.

Pattern 6: Fine-Grained Tracker (Column Lineage)

❖ The Problem

- You need to deprecate the column `user_email` for compliance. Which downstream reports actually use this specific column? Dataset lineage is too broad (it only shows table dependencies).

❖ The Solution

- Track lineage at the Column or Row level.
- Column Lineage: Parse query plans to see that `output.full_name` is derived from `input.first_name + input.last_name`.
- Row Lineage: Add metadata headers (e.g., `Job_ID`, `Source_Batch_ID`) to every row to trace individual records back to their origin.

Summary – The Observability Pillars

❖ Data Detection:

- **Flow Interruption:** Is data arriving?
- **Skew:** Is the volume correct?

❖ Time Detection:

- **Lag:** Are we keeping up with the stream?
- **SLA Misses:** Are we delivering on time?

❖ Tracking:

- **Dataset Tracker:** Who produces this table?
- **Fine-Grained Tracker:** Where does this column come from?

Conclusions

- ❖ Decouple Everything: Storage should be separate from Compute; Ingestion should be separate from Transformation.
- ❖ Idempotency is King: Systems must be able to retry failures without duplicating data or corrupting state.
- ❖ Treat Data as a Product: Apply software engineering rigor (version control, testing, CI/CD) to data assets.
- ❖ Design for Failure: Use patterns like Dead-Letter Queues and Circuit Breakers because failures will happen.

Conclusions

- ❖ **Simplicity Wins:** The trend is moving back towards simpler architectures (e.g., Lakehouse, Zero-ETL where appropriate) to reduce maintenance burden.
- ❖ **Convergence:** The gap between Batch and Streaming is closing (Unified Processing).
- ❖ **Next Steps:**
 - Audit your current pipelines against these patterns.
 - Identify one "Technical Debt" area (e.g., Small Files, Hardcoded Secrets) and refactor it using a pattern learned today.
- ❖ **Final Thought:** "Good architecture is not about using the newest tool, but about choosing the right pattern for the problem."